

1 Bitwise Trick

1.1 tricks

```

1 bool isOdd(int x) {
2     return x & 1;
3 }
4
5 int isolateRightmostSetBit(int x) {
6     return x & -x; // 取得最右邊的 1
7 }
8
9 bool isPowerOfTwo(int x) {
10    return x > 0 && (x & (x - 1)) == 0;
11 }
12
13 int countSetBits(int x) {
14     int count = 0;
15     while (x) {
16         x &= (x - 1); // 清除最右邊的 1
17         count++;
18     }
19     return count;
20 }
21
22 int turnOffRightmostSetBit(int x) {
23     return x & (x - 1); // 將最右邊的 1 設為 0
24 }
25
26 int setBit(int x, int pos) {
27     return x | (1 << pos); // 設該位元為 1
28 }
29
30 int clearBit(int x, int pos) {
31     return x & ~(1 << pos); // 設該位元為 0
32 }
33
34 int toggleBit(int x, int pos) {
35     return x ^ (1 << pos); // 切換該位元
36 }
37
38 int getBit(int x, int pos) {
39     return (x >> pos) & 1; // 取得該位元值
40 }
41
42 int modPowerOfTwo(int x, int mod) {
43     return x & (mod - 1); // x % mod, mod 必須
44     // 是 2 的幕次
45 }
46
47 // a + b = (a ^ b) + 2 * (a & b)
48 int add(int a, int b) {
49     while (b != 0) {
50         int carry = a & b; // 計算進位
51         a = a ^ b; // 無進位相加
52         b = carry << 1; // 將進位左移一位
53     }
54     return a;
55 }
56
57 // a - b = (a ^ b) - 2 * ((~a) & b)
58 int subtract(int a, int b) {

```

```

57 while (b != 0) {
58     int borrow = (~a) & b; // 計算借位
59     a = a ^ b; // 無借位相減
60     b = borrow << 1; // 將借位左移一位
61 }
62 return a;
63 }
64 // a * b = sum(a << i) for each set bit i in b
65 int multiply(int a, int b) {
66     int result = 0;
67     while (b > 0) {
68         if (b & 1) { // 如果 b 的最低位是 1
69             result = add(result, a); // 使用
70             // 上面的加法函數
71         }
72         a <<= 1; // 將 a 左移一位 (相當於乘以 2)
73         b >>= 1; // 將 b 右移一位 (相當於除以 2)
74     }
75     return result;
76 }
77 // a / b = sum(1 << i) for each set bit i in a
78 int divide(int a, int b) {
79     if (b == 0) throw std::invalid_argument("Division by zero");
80     int result = 0;
81     int power = 1;
82     int tempB = b;
83
84     while (tempB <= a && tempB > 0) { // 防止溢位
85         tempB <<= 1;
86         power <<= 1;
87     }
88     while (power > 1) {
89         tempB >>= 1;
90         power >>= 1;
91         if (a >= tempB) {
92             a = subtract(a, tempB); // 使用
93             // 上面的減法函數
94             result = add(result, power); // 使用上面的加法函數
95         }
96     }
97     return result;
98 }
99 int findUnique(const std::vector<int>& nums) {
100    {
101        int unique = 0;
102        for (int num : nums) {
103            unique ^= num; // XOR 相同的數字會抵消
104        }
105        return unique;
106    }
107 }
108
109 int graycode(int n) { // n 轉 Gray code

```

2 DP

2.1 LIS

```

11 LISLength(const vector<ll>& arr) {
12     vector<ll> d;
13     for (ll x : arr) {
14         auto it = lower_bound(d.begin(), d.end(), x);
15         if (it == d.end()) d.push_back(x);
16         else *it = x;
17     }
18     return d.size();
19 }
20
21 // find LIS sequence
22 for (ll i = 0; i < num.size(); ++i) {
23     if (lis.empty() || lis.back() < num[i]) {
24         lis.push_back(num[i]);
25         dp[i] = lis.size();
26     }
27     else {
28         auto iter = lower_bound(all(lis), num[i]);
29         dp[i] = iter - lis.begin() + 1;
30         *iter = num[i];
31     }
32 }
33 ll length = lis.size();
34 for (ll i = num.size() - 1; i >= 0; --i) {
35     if (dp[i] == length) {
36         ans.push_back(num[i]);
37         length--;
38     }
39 }

```

2.2 Knapsack

```

1 // 0/1 knapsack
2
3 const ll INF = 9e18;
4
5 ll n, wmx; cin >> n >> wmx;
6 vector<ll> w(n + 1); for (ll i = 1; i <= n; ++i) cin >> w[i];
7 vector<ll> v(n + 1); for (ll i = 1; i <= n; ++i) cin >> v[i];
8
9 vector<ll> dp(wmx + 1, -INF); dp[0] = 0;
10 for (ll i = 1; i <= n; ++i) {
11     for (ll j = wmx; j >= w[i]; --j) {
12         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
13     }
14 }
15

```

```

16 ll mx = dp[0];
17 for (ll j = 1; j <= wmx; ++j) {
18     mx = max(mx, dp[j]);
19 }
20 cout << mx << '\n';
21
22 // -----
23
24 // Unbounded
25 // just change the inner for loop to the following line
26 // for (ll j = w[i]; j <= wmx; ++j) ...

```

2.3 Subset DP

```

1 for (ll s = 1; s < (1LL << n); ++s) {
2     for (ll sbs = s; sbs > 0; sbs = ((sbs - 1) & s)) {
3         ll c = s - sbs; // ll c = sbs ^ s; // this one is equivalent
4     }
5 }
6 // O(3^n), n <= 16
7 // sbs 是降序的 s 的子集
8 // c 是升序的 s 的子集

```

2.4 Interval DP

```

1 ll n; // the length of the array
2
3 vector<ll> a(n + 1);
4
5 vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
6
7 for (ll i = 1; i <= n; ++i) {
8     dp[i][i] = "initial value"; // Length 1 interval [i, i]
9 }
10 for (ll len = 2; len <= n; ++len) { // Loop through all length from 2 to n
11     for (ll l = 1; l <= n - len + 1; ++l) {
12         ll r = l + len - 1;
13
14         dp[l][r] = ...; // interval [l, r]
15     }
16 }
17
18 dp[1][n] // the answer (interval [1, n])

```

2.5 LCS

```

1 ll n; cin >> n;
2 string s; cin >> s; s = " " + s;
3 ll m; cin >> m;
4 string t; cin >> t; t = " " + t;

```

```

5 vector<vector<ll>> dp(n + 1, vector<ll>(m +
6 1, 0));
7 for (ll i = 1; i <= n; ++i) {
8     for (ll j = 1; j <= m; ++j) {
9         if (s[i] == t[j]) {
10             dp[i][j] = dp[i - 1][j - 1] + 1;
11         }
12         else {
13             dp[i][j] = max(dp[i - 1][j], dp[i][j -
14 1]);
15         }
16     }
17 }
18 cout << dp[n][m] << '\n'; // the length of
    LCS

```

2.6 Digit DP

```

1 string num;
2
3 ll dp[the max len of num + 1][2][2][...];
4 memset(dp, -1, sizeof(dp));
5
6 ll count(ll pos, bool tight, bool
7 leadingZero, ...) {
8     if (dp[pos][tight][leadingZero][...] !=
9     -1) {
10         return dp[pos][tight][leadingZero
11 ][...];
12     }
13
14     if (base-case) { // e.g. pos == num.
15         size() or other base case condition
16         // do something
17     }
18
19     ll res = 0;
20     ll up = (tight ? num[pos] - '0' : 9);
21     for (ll d = 0; d <= up; ++d) {
22         res += count(
23             pos + 1,
24             tight and (d == num[pos] - '0'),
25             leadingZero and (d == 0),
26             ...
27         );
28     }
29
30     return dp[pos][tight][leadingZero][...]
31     = res;
32 }
33
34 count(0, true, true, ...) // the answer
35
36 // =====
37 // =====
38
39 // AtCoder ABC154E
40 // Find the number of integers between 1 and
41 // N (inclusive)
42 // that contains exactly K non-zero digits
43 // when written in base ten.

```

```

37 // 1 <= N <= 10^100, 1 <= K <= 3
38 #include <bits/stdc++.h>
39 using namespace std;
40 using ll = long long;
41
42 ll k;
43 string num;
44
45 ll dp[102][2][4];
46
47 ll count(ll pos, bool tight, ll cnt) {
48     if (dp[pos][tight][cnt] != -1) {
49         return dp[pos][tight][cnt];
50     }
51
52     if (cnt == 0) return dp[pos][tight][cnt]
53     = 1;
54     else if (pos == num.size()) return dp[
55     pos][tight][cnt] = 0;
56
57     ll res = 0;
58     ll up = (tight ? num[pos] - '0' : 9);
59     for (ll d = 0; d <= up; ++d) {
60         res += count(
61             pos + 1,
62             tight and (d == num[pos] - '0'),
63             cnt + (d == 0 ? 0 : -1)
64         );
65     }
66
67     return dp[pos][tight][cnt] = res;
68 }
69
70 int main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73
74     cin >> num >> k;
75
76     memset(dp, -1, sizeof(dp));
77
78     cout << count(0, true, k) << '\n';
79
80     return 0;

```

3 D&C

3.1 MergeSort Finds the Number of Inversions

```

1 void merge(vector<int>& arr, ll l, ll r, ll
2 x, ll y) {
3     ll left = l;
4     vector<ll> tmp; tmp.reserve(y - l + 1);
5     while (l <= r and x <= y) {
6         if (arr[l] <= arr[x]) {
7             tmp.push_back(arr[l++]);

```

```

8     else {
9         tmp.push_back(arr[x++]);
10    }
11    }
12    while (l <= r) {
13        tmp.push_back(arr[l++]);
14    }
15    while (x <= y) {
16        tmp.push_back(arr[x++]);
17    }
18    for (ll i = left; i <= y; ++i) {
19        arr[i] = tmp[i - left];
20    }
21 }
22 ll mergeSort(vector<ll>& arr, ll l, ll r) {
23     if (l == r) return 0;
24     ll mid = l + (r - l)/2;
25     ll lcnt = mergeSort(arr, l, mid);
26     ll rcnt = mergeSort(arr, mid + 1, r);
27
28     // ----- main logic -----
29     ll cnt = lcnt + rcnt;
30     ll a = l, b = mid, c = mid + 1, d = r;
31     // c is the current checking
32     // position
33     while (a <= b) {
34         while (c <= d and arr[a] > arr[c]) c
35         += 1;
36         cnt += c - (mid + 1);
37         a += 1;
38     }
39     // -----
40     merge(arr, l, mid, mid + 1, r);
41     return cnt;

```

4 Data Structure

4.1 Segment Tree

```

1 // CSES Range Updates and Sums
2 // 1. Increase each value in range [a,b] by
3 // x.
4 // 2. Set each value in range [a,b] to x.
5 // 3. Calculate the sum of values in range [
6 // a,b].
7
8 #include <bits/stdc++.h>
9 using namespace std;
10 using ll = long long;
11
12 #define lson 2*n + 1
13 #define rson 2*n + 2
14
15 class Node {
16 public:
17     ll val;
18     ll setVal;
19     ll addVal;

```

```

18     bool isSet;
19 };
20
21 ll sz, q;
22 vector<ll> a;
23 vector<Node> nds;
24
25 void build(ll l, ll r, ll n = 0) {
26     if (l == r) {
27         nds[n].val = a[l];
28         nds[n].setVal = 0;
29         nds[n].addVal = 0;
30         nds[n].isSet = false;
31         return;
32     }
33     ll mid = l + (r - l)/2;
34     build(l, mid, lson);
35     build(mid + 1, r, rson);
36     nds[n].val = nds[lson].val + nds[rson].
37     val;
38 }
39
40 void push(ll l, ll r, ll n) {
41     ll mid = l + (r - l)/2;
42
43     if (nds[n].isSet) {
44         nds[lson].val = nds[n].setVal*(mid -
45         l + 1);
46         nds[lson].setVal = nds[n].setVal;
47         nds[lson].addVal = 0;
48         nds[lson].isSet = true;
49
50         nds[rson].val = nds[n].setVal*(r - (
51         mid + 1) + 1);
52         nds[rson].setVal = nds[n].setVal;
53         nds[rson].addVal = 0;
54         nds[rson].isSet = true;
55
56         nds[n].isSet = false;
57     }
58
59     nds[lson].val += nds[n].addVal*(mid - l
60     + 1);
61     nds[lson].addVal += nds[n].addVal;
62
63     nds[rson].val += nds[n].addVal*(r - (mid
64     + 1) + 1);
65     nds[rson].addVal += nds[n].addVal;
66
67     nds[n].addVal = 0;
68 }
69
70 void setVal(ll x, ll y, ll val, ll l, ll r,
71 ll n = 0) {
72     if (l == x and r == y) {
73         nds[n].val = val*(y - x + 1);
74         nds[n].setVal = val;
75         nds[n].addVal = 0;
76         nds[n].isSet = true;
77         return;
78     }
79     ll mid = l + (r - l)/2;
80     if (y <= mid) {
81         setVal(x, y, val, l, mid, lson);

```

```

78 } else if (x >= mid + 1) {
79     setVal(x, y, val, mid + 1, r, rson);
80 } else {
81     setVal(x, mid, val, l, mid, lson);
82     setVal(mid + 1, y, val, mid + 1, r, rson);
83 }
84 nds[n].val = nds[lson].val + nds[rson].val;
85 }
86
87 void addVal(ll x, ll y, ll val, ll l, ll r, ll n = 0) {
88     if (l == x and r == y) {
89         nds[n].val += val*(y - x + 1);
90         nds[n].addVal += val;
91         return;
92     }
93     push(l, r, n);
94     ll mid = l + (r - l)/2;
95     if (y <= mid) {
96         addVal(x, y, val, l, mid, lson);
97     } else if (x >= mid + 1) {
98         addVal(x, y, val, mid + 1, r, rson);
99     } else {
100         addVal(x, mid, val, l, mid, lson);
101         addVal(mid + 1, y, val, mid + 1, r, rson);
102     }
103     nds[n].val = nds[lson].val + nds[rson].val;
104 }
105
106 ll query(ll x, ll y, ll l, ll r, ll n = 0) {
107     if (l == x and r == y) return nds[n].val;
108     ;
109     push(l, r, n);
110     ll mid = l + (r - l)/2;
111     if (y <= mid) {
112         return query(x, y, l, mid, lson);
113     } else if (x >= mid + 1) {
114         return query(x, y, mid + 1, r, rson);
115     } else {
116         ll a = query(x, mid, l, mid, lson);
117         ll b = query(mid + 1, y, mid + 1, r, rson);
118         return a + b;
119     }
120 }
121
122 int main() {
123     ios::sync_with_stdio(false);
124     cin.tie(nullptr);
125
126     cin >> sz >> q;
127
128     a.resize(sz + 1);
129     for (ll i = 1; i <= sz; ++i) cin >> a[i];
130
131     nds.resize(4*sz);
132     build(1, sz);
133
134     while (q--) {
135         ll act; cin >> act;

```

```

135         if (act == 1) {
136             ll l, r, val; cin >> l >> r >> val;
137             addVal(l, r, val, 1, sz);
138         } else if (act == 2) {
139             ll l, r, val; cin >> l >> r >> val;
140             setVal(l, r, val, 1, sz);
141         } else if (act == 3) {
142             ll l, r; cin >> l >> r;
143             cout << query(l, r, 1, sz) << '\n';
144         }
145     }
146     return 0;
147 }
148

```

4.2 BIT

```

1 inline ll lowbit(ll x) {
2     return x & -x;
3 }
4
5 ll n;
6 vector<ll> a, bit;
7
8 void add(ll pos, ll val) {
9     for (ll i = pos; i <= n; i += lowbit(i))
10         bit[i] += val;
11 }
12
13 void init() {
14     for (ll i = 1; i <= n; ++i) {
15         add(i, a[i]);
16     }
17 }
18
19 ll query(ll pos) { // [1, pos]
20     ll sum = 0;
21     for (ll i = pos; i >= 1; i -= lowbit(i))
22         sum += bit[i];
23 }
24
25 return sum;
26 }

```

4.3 Treap

```

1 // CSES Cut and Paste
2 // Given a string, your task is to process operations
3 // where you cut a substring and paste it to the end of the string.
4 // What is the final string after all the operations?
5
6 #include <bits/stdc++.h>

```

```

7 using namespace std;
8 using ll = long long;
9 using pll = pair<ll, ll>;
10 template<class T> using vec = vector<T>;
11 #define ff first
12 #define ss second
13 #define pb push_back
14 #define rep(n) for (ll _ = 1; _ <= n; ++_)
15
16 //
17
18 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
19
20 class Node {
21 public:
22     Node* l, * r;
23     ll val, pri, sz;
24     Node(ll _val) {
25         l = r = nullptr;
26         val = _val;
27         pri = rng();
28         sz = 1;
29     }
30     void pull() {
31         sz = 1;
32         if (l) sz += l->sz;
33         if (r) sz += r->sz;
34     }
35 };
36
37 Node* merge(Node* a, Node* b) {
38     if (!a || !b) return a ? a : b;
39     if (a->pri < b->pri) {
40         a->r = merge(a->r, b);
41         a->pull();
42         return a;
43     } else {
44         b->l = merge(a, b->l);
45         b->pull();
46         return b;
47     }
48 }
49
50 void splsz(Node* T, Node*& a, Node*& b, ll s) {
51     if (!T) {
52         a = b = nullptr;
53         return;
54     }
55
56     ll lsz = T->l ? T->l->sz : 0;
57     if (lsz + 1 <= s) {
58         a = T;
59         splsz(T->r, a->r, b, s - (lsz + 1));
60         a->pull();
61     } else {
62         b = T;
63         splsz(T->l, a, b->l, s);
64         b->pull();
65     }
66 }
67

```

```

68 void move_to_tail(Node*& root, ll l, ll r) {
69     // 1-indexed
70     Node* left, * mid, * right;
71     splsz(root, left, mid, l - 1);
72     splsz(mid, mid, right, r - l + 1);
73     root = merge(left, right);
74     root = merge(root, mid);
75 }
76
77 void inorder(Node* root, string& res) {
78     if (!root) return;
79     inorder(root->l, res);
80     res += char(root->val);
81     inorder(root->r, res);
82 }
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     ll n, q;
89     cin >> n >> q;
90
91     string s;
92     cin >> s;
93
94     Node* root = nullptr;
95     for (ll i = 0; i < n; ++i) {
96         root = merge(root, new Node(s[i]));
97     }
98
99     rep (q) {
100         ll l, r;
101         cin >> l >> r;
102         move_to_tail(root, l, r);
103     }
104
105     string res = "";
106     inorder(root, res);
107     cout << res << '\n';
108
109     return 0;
110 }
111
112 //
113 // =====
114 // CSES Substring Reversals
115 // Given a string, your task is to process operations
116 // where you reverse a substring of the string.
117 // What is the final string after all the operations?
118
119 #include <bits/stdc++.h>
120 using namespace std;
121 using ll = long long;
122 using pll = pair<ll, ll>;
123 template<class T> using vec = vector<T>;
124 #define ff first
125 #define ss second

```

```

126 #define pb push_back
127 #define rep(n) for (ll _ = 1; _ <= n; ++_)
128 //
129 -----
130 mt19937 rng(chrono::steady_clock::now().
131             time_since_epoch().count());
132
133 class Node {
134 public:
135     Node* l,* r;
136     ll val, pri, sz;
137     bool rev_tag;
138     Node(ll _val) {
139         l = r = nullptr;
140         val = _val;
141         pri = rng();
142         sz = 1;
143         rev_tag = false;
144     }
145     void pull() {
146         sz = 1;
147         if (l) sz += l->sz;
148         if (r) sz += r->sz;
149     }
150     void push() {
151         if (rev_tag) {
152             swap(l, r);
153             if (l) l->rev_tag = !l->rev_tag;
154             if (r) r->rev_tag = !r->rev_tag;
155             rev_tag = false;
156         }
157     }
158 };
159
160 Node* merge(Node* a, Node* b) {
161     if (!a || !b) return a ? a : b;
162     if (a->pri < b->pri) {
163         a->push();
164         a->r = merge(a->r, b);
165         a->pull();
166         return a;
167     } else {
168         b->push();
169         b->l = merge(a, b->l);
170         b->pull();
171         return b;
172     }
173 }
174
175 void splsz(Node* T, Node*& a, Node*& b, ll s
176 ) {
177     if (!T) {
178         a = b = nullptr;
179         return;
180     }
181     T->push();
182     ll lsr = T->l ? T->l->sz : 0;
183     if (lsr + 1 <= s) {
184         a = T;
185         splsz(T->r, a->r, b, s - (lsr + 1));
186         a->pull();
187     } else {
188         b = T;
189         splsz(T->l, a, b->l, s);
190         b->pull();
191     }
192 }
193
194 void rev_range(Node*& root, ll l, ll r) {
195     // 1-indexed
196     Node* left,* mid,* right;
197     splsz(root, left, mid, l - 1);
198     splsz(mid, mid, right, r - l + 1);
199     mid->push(); // this line isn't
200                 necessary
201     if (mid) mid->rev_tag = !mid->rev_tag;
202     root = merge(left, mid);
203     root = merge(root, right);
204 }
205
206 void inorder(Node* root, string& res) {
207     if (!root) return;
208     root->push();
209     inorder(root->l, res);
210     res += char(root->val);
211     inorder(root->r, res);
212 }
213
214 int main() {
215     ios::sync_with_stdio(false);
216     cin.tie(nullptr);
217
218     ll n, q;
219     cin >> n >> q;
220
221     string s;
222     cin >> s;
223
224     Node* root = nullptr;
225     for (ll i = 0; i < n; ++i) {
226         root = merge(root, new Node(s[i]));
227     }
228
229     rep (q) {
230         ll l, r;
231         cin >> l >> r;
232         rev_range(root, l, r);
233     }
234
235     string res = "";
236     inorder(root, res);
237     cout << res << '\n';
238
239     return 0;
240 }
241
242 // CSES Reversals and Sums
243 // Given an array of n integers, you have to
244 // process following operations:
245 // reverse a subarray
246 // calculate the sum of values in a subarray
247
248 #include <bits/stdc++.h>
249 using namespace std;
250 using ll = long long;
251 using pll = pair<ll, ll>;
252 template<class T> using vec = vector<T>;
253 #define endl '\n'
254 #define ff first
255 #define ss second
256 #define pb push_back
257 #define rep(n) for (ll _ = 1; _ <= n; ++_)
258 //
259 -----
260 mt19937 rng(chrono::steady_clock::now().
261             time_since_epoch().count());
262
263 class Node {
264 public:
265     Node* l,* r;
266     ll val, pri, sz;
267     ll sum;
268     bool rev_tag;
269     Node(ll _val) {
270         l = r = nullptr;
271         val = _val;
272         pri = rng();
273         sz = 1;
274         sum = _val;
275         rev_tag = false;
276     }
277     void pull() {
278         sz = 1;
279         if (l) sz += l->sz;
280         if (r) sz += r->sz;
281
282         sum = val;
283         if (l) sum += l->sum;
284         if (r) sum += r->sum;
285     }
286     void push() {
287         if (rev_tag) {
288             swap(l, r);
289             if (l) l->rev_tag = !l->rev_tag;
290             if (r) r->rev_tag = !r->rev_tag;
291             rev_tag = false;
292         }
293     }
294
295     Node* merge(Node* a, Node* b) {
296         if (!a || !b) return a ? a : b;
297         if (a->pri < b->pri) {
298             a->push();
299             a->r = merge(a->r, b);
300             a->pull();
301             return a;
302         } else {
303             b->push();
304             b->l = merge(a, b->l);
305             b->pull();
306             return b;
307         }
308     }
309 };
310
311 void splsz(Node* root, Node*& a, Node*& b,
312           ll s) {
313     if (!root) {
314         a = b = nullptr;
315         return;
316     }
317     root->push();
318     ll lsr = root->l ? root->l->sz : 0;
319     if (lsr + 1 <= s) {
320         a = root;
321         splsz(root->r, a->r, b, s - (lsr +
322                                     1));
323     } else {
324         b = root;
325         splsz(root->l, a, b->l, s);
326         b->pull();
327     }
328 }
329
330 void rev_range(Node*& root, ll l, ll r) {
331     // 1-indexed
332     Node* left,* mid,* right;
333     splsz(root, left, mid, l - 1);
334     splsz(mid, mid, right, r - l + 1);
335     if (mid) mid->rev_tag = !mid->rev_tag;
336     root = merge(left, mid);
337     root = merge(root, right);
338 }
339
340 ll qry_range_sum(Node*& root, ll l, ll r) {
341     Node* left,* mid,* right;
342     splsz(root, left, mid, l - 1);
343     splsz(mid, mid, right, r - l + 1);
344     ll res = 0;
345     if (mid) res = mid->sum;
346     root = merge(left, mid);
347     root = merge(root, right);
348     return res;
349 }
350
351 int main() {
352     ios::sync_with_stdio(false);
353     cin.tie(nullptr);
354
355     ll n, q;
356     cin >> n >> q;
357
358     Node* root = nullptr;
359     for (ll i = 0; i < n; ++i) {
360         ll val; cin >> val;
361         root = merge(root, new Node(val));
362     }
363
364     rep (q) {
365         ll act, l, r;
366         cin >> act >> l >> r;
367         if (act == 1) {
368             rev_range(root, l, r);
369         } else if (act == 2) {
370             cout << qry_range_sum(root, l, r
371                                     ) << endl;
372         }
373     }
374
375     return 0;
376 }

```

```

372 }
373
374 //
=====
375 //
=====
376
377 // SPOJ ORDERSET
378 // In this problem, you have to maintain a
dynamic set of numbers
379 // which support the two fundamental
operations
380 // INSERT(S,x): if x is not in S, insert
x into S
381 // DELETE(S,x): if x is in S, delete x
from S
382 // and the two type of queries
383 // K-TH(S) : return the k-th smallest
element of S
384 // COUNT(S,x): return the number of
elements of S smaller than x
385
386 #include <bits/stdc++.h>
387 using namespace std;
388 using ll = long long;
389 using pll = pair<ll, ll>;
390 template<class T> using vec = vector<T>;
391 #define endl '\n'
392 #define ff first
393 #define ss second
394 #define pb push_back
395 #define rep(n) for (ll _ = 1; _ <= n; ++_)
396
397 //
-----
398
399 mt19937 rng(chrono::steady_clock::now().
time_since_epoch().count());
400
401 class Node {
402 public:
403     Node* l,* r;
404     ll val, pri, sz;
405     Node(ll _val) {
406         l = r = nullptr;
407         val = _val;
408         pri = rng();
409         sz = 1;
410     }
411     void pull() {
412         sz = 1;
413         if (l) sz += l->sz;
414         if (r) sz += r->sz;
415     }
416 };
417
418 Node* merge(Node* a, Node* b) {
419     if (!a || !b) return a ? a : b;
420     if (a->pri < b->pri) {
421         a->r = merge(a->r, b);
422         a->pull();
423         return a;
424     } else {
425         b->l = merge(a, b->l);
426         b->pull();
427         return b;
428     }
429 }
430
431 void split(Node* root, Node*& a, Node*& b,
ll k) {
432     if (!root) {
433         a = b = nullptr;
434         return;
435     }
436     if (root->val <= k) {
437         a = root;
438         split(root->r, a->r, b, k);
439         a->pull();
440     } else {
441         b = root;
442         split(root->l, a, b->l, k);
443         b->pull();
444     }
445 }
446
447 void splsz(Node* root, Node*& a, Node*& b,
ll s) {
448     if (!root) {
449         a = b = nullptr;
450         return;
451     }
452     ll lsr = root->l ? root->l->sz : 0;
453     if (lsr + 1 <= s) {
454         a = root;
455         splsz(root->r, a->r, b, s - (lsr +
1));
456         a->pull();
457     } else {
458         b = root;
459         splsz(root->l, a, b->l, s);
460         b->pull();
461     }
462 }
463
464 ll count(Node*& root, ll k) {
465     Node* left,* mid,* right;
466     split(root, left, mid, k - 1);
467     split(mid, mid, right, k);
468     ll res = 0;
469     if (mid) res = mid->sz;
470     root = merge(left, mid);
471     root = merge(root, right);
472     return res;
473 }
474
475 ll count_lt(Node*& root, ll k) { // count
less than
476     Node* left,* right;
477     split(root, left, right, k - 1);
478     ll res = 0;
479     if (left) res = left->sz;
480     root = merge(left, right);
481     return res;
482 }
483
484 ll kth_smallest(Node*& root, ll k) {
485     assert(root && 1 <= k && k <= root->sz);
486     Node* left,* mid,* right;
487     splsz(root, left, mid, k - 1);
488     splsz(mid, mid, right, 1);
489     ll res = mid->val;
490     root = merge(left, mid);
491     root = merge(root, right);
492     return res;
493 }
494
495 void insert(Node*& root, ll k) {
496     Node* left,* right;
497     split(root, left, right, k);
498     Node* a = new Node(k);
499     root = merge(left, a);
500     root = merge(root, right);
501 }
502
503 void remove_multi(Node*& root, ll k) {
504     Node* left,* mid,* right;
505     split(root, left, mid, k - 1);
506     split(mid, mid, right, k);
507     root = merge(left, right);
508 }
509
510 void remove_single(Node*& root, ll k) {
511     Node* left,* mid,* right;
512     split(root, left, mid, k - 1);
513     splsz(mid, mid, right, 1);
514     if (!mid || mid->val != k) {
515         root = merge(left, mid);
516         root = merge(root, right);
517     } else {
518         root = merge(left, right);
519     }
520 }
521
522 int main() {
523     ios::sync_with_stdio(false);
524     cin.tie(nullptr);
525
526     // The element can be duplicated in the
treap
// of the implementation of this program
527
528     // But the element cannot be duplicated
in this problem
529     // (Use "count" to check whether the
element exists)
530
531     ll q; cin >> q;
532
533     Node* root = nullptr;
534
535     rep (q) {
536         char act; cin >> act;
537         ll k; cin >> k;
538         if (act == 'I') {
539             if (count(root, k) == 0) {
540                 insert(root, k);
541             }
542         } else if (act == 'D') {
543             remove_single(root, k);
544         } else if (act == 'K') {
545             if (!root || k > root->sz) {
546                 cout << "invalid" << endl;
547             } else {
548                 cout << kth_smallest(root, k)
<< endl;
549             }
550         }
551     }
552     return 0;
553 }

```

4.4 Sparse Table

```

1 // if the max size of arr is 200000
2 vector<ll> arr;
3
4 // -----
5
6 // Sparse Table
7 const ll lgmx = 17; // floor(Log2(200000))
8 ll rmq[lgmx + 1][200000 + 1];
9 void init(ll n) { // O(nLogn)
10     for (ll i = 1; i <= n; ++i) rmq[0][i] =
arr[i];
11     for (ll h = 1; h <= lgmx; ++h)
12         for (ll i = 1; i + (1 << h) - 1 <= n
; ++i)
13             rmq[h][i] = min(rmq[h - 1][i],
rmq[h - 1][i + (1 << (h - 1))]);
14 }
15 ll flg ull x) {return 63 - __builtin_clzll(x
);}
16 ll query(ll l, ll r) { // O(1)
17     ll h = flg(r - l + 1);
18     return min(rmq[h][l], rmq[h][r - (1 << h
) + 1]);
19 }
20
21 // -----
22
23 // initialize the array
24 ll n; cin >> n;
25 arr.resize(n + 1);
26 for (ll i = 1; i <= n; ++i) cin >> arr[i];
27
28 // initialize the sparse table
29 init(n);

```

4.5 DSU

```

1 ll cc;
2 vector<ll> djs, sz;
3 ll find(ll u) {
4     if (u == djs[u]) return u;
5     return djs[u] = find(djs[u]);
6 }
7 void join(ll u, ll v) {
8     u = find(u);

```

```

9   v = find(v);
10  if (u == v) return; // don't forgot
    this line
11  if (sz[u] < sz[v]) swap(u, v);
12  djs[v] = u;
13  sz[u] += sz[v];
14  cc -= 1;
15 }
16 void init(ll n) {
17     djs.clear(); djs.resize(n + 1);
18     for (ll i = 1; i <= n; ++i) djs[i] = i;
19     sz.clear(); sz.resize(n + 1, 1);
20     cc = n;
21 }

```

5 Geometry

5.1 Line Segment Intersection Test

```

1 bool isSegInter(const vec& a, const vec& b,
2               const vec& c, const vec& d) {
3     ll ori1 = ori(a, b, c);
4     ll ori2 = ori(a, b, d);
5     ll ori3 = ori(c, d, a);
6     ll ori4 = ori(c, d, b);
7     if (isCollinear(a, b, c) && isCollinear(
8         a, b, d)) {
9         return isOnSeg(a, b, c) || isOnSeg(a,
10            b, d) ||
11            isOnSeg(c, d, a) || isOnSeg(c,
12                d, b);
13     }
14     return ori1 * ori2 <= 0 && ori3 * ori4
15        <= 0;
16 }

```

5.2 Simple Polygon Area

```

1 // pts = {p0, p1, ... pn-1, p0}, 0-based
2 // for simple polygon area
3 ll shoelace2(const vector<vec> &pts) {
4     ll n = pts.size() - 2, res = 0;
5     for (ll i = 0; i <= n; ++i) {
6         res += pts[i].x * pts[i + 1].y;
7         res -= pts[i].y * pts[i + 1].x;
8     }
9     return abs(res);
10 }

```

5.3 Vector

```

1 class vec {
2 public:
3     ll x, y;
4     vec() {}

```

```

5     vec(ll _x, ll _y) : x(_x), y(_y) {}
6     vec operator+(const vec& v) const {
7         return vec(this->x + v.x, this->y +
8             v.y);
9     }
10    vec operator-(const vec& v) const {
11        return vec(this->x - v.x, this->y -
12            v.y);
13    }
14    ll operator*(const vec& v) const {
15        return this->x * v.x + this->y * v.y;
16    }
17    ll operator^(const vec& v) const {
18        return this->x * v.y - this->y * v.x;
19    }
20    bool operator<(const vec& v) const {
21        if (this->x != v.x) return this->x <
22            v.x;
23        return this->y < v.y;
24    }
25    vec& operator=(const vec& v) {
26        this->x = v.x;
27        this->y = v.y;
28        return *this;
29    }
30 }
31 ll sign(ll x) {
32     if (x == 0) return 0;
33     if (x < 0) return -1;
34     return 1;
35 }
36 ll ori(const vec& o, const vec& a, const vec&
37     b) {
38     return sign((a - o) ^ (b - o));
39 }
40 bool isCollinear(const vec& a, const vec& b,
41     const vec& c) {
42     return ori(a, b, c) == 0;
43 }
44 bool isOnSeg(const vec& a, const vec& b,
45     const vec& p) {
46     return isCollinear(a, b, p) && sign((p -
47         a) * (p - b)) <= 0;
48 }

```

5.4 Convex Hull

```

1 // pts = {p0, p1, ... pn-1}, 0-based
2 // the points in pts should be distinct
3 vector<vec> convexHull(const vector<vec> &
4     _pts) {
5     vector<vec> pts = _pts;
6     sort(pts.begin(), pts.end());
7
8     ll n = pts.size();
9     vector<vec> hull(1, pts[0]);
10    for (ll i = 1; i < n; ++i) {

```

```

11        while (hull.size() >= 2 &&
12            ori(hull[hull.size() - 2],
13                hull.back(), pts[i]) < 0)
14            hull.pop_back();
15        hull.push_back(pts[i]);
16    }
17    ll m = hull.size();
18    for (ll i = n - 2; i >= 0; --i) {
19        while (hull.size() - m + 1 >= 2 &&
20            ori(hull[hull.size() - 2],
21                hull.back(), pts[i]) < 0)
22            hull.pop_back();
23        hull.push_back(pts[i]);
24    }
25    hull.pop_back();
26    return hull;
27 }

```

6 Graph

6.1 Tarjan

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }

```

```

34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     cin >> n >> m;
41     inStack.resize(n + 1);
42     in.resize(n + 1, -1);
43     low.resize(n + 1);
44     scc.resize(n + 1);
45     adj.resize(n + 1);
46     for (ll i = 1; i <= m; ++i) {
47         ll u, v; cin >> u >> v;
48         adj[u].pb(v);
49     }
50
51     for (ll u = 1; u <= n; ++u) {
52         if (in[u] != -1) continue;
53         dfs(u);
54     }
55
56     cout << sccCnt << '\n';
57     for (ll u = 1; u <= n; ++u) {
58         cout << scc[u] << ' ';
59     } cout << '\n';
60
61     return 0;
62 }

```

6.2 Bridge

```

1 void dfs(ll u, ll pa = -1) {
2     low[u] = in[u] = ++t;
3     for (ll v : adj[u]) {
4         if (v == pa) continue;
5         if (in[v] == -1) {
6             dfs(v, u);
7             low[u] = min(low[u], low[v]);
8             if (low[v] > in[u]) "edge (u, v)
9                 is bridge" // find bridge
10        } else if (in[v] < in[u]) {
11            low[u] = min(low[u], in[v]);
12        }
13    }

```

6.3 Bellman Ford Detects Negative Cycle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define ff first
6 #define ss second
7 #define pb push_back
8 #define rep(n) for (ll _ = 1; _ <= n; ++_)
9

```



```

10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     const ll INF = 5e12 + 1000;
15
16     ll n, m; cin >> n >> m;
17     vector<ll> dis(n + 1, INF);
18     vector<vector<pll>> adj(n + 1);
19     rep(m) {
20         ll u, v, w; cin >> u >> v >> w;
21         adj[u].pb(pll(w, v));
22     }
23     for (ll i = 1; i <= n; ++i) {
24         adj[0].pb(pll(0, i));
25     }
26
27     dis[0] = 0;
28     rep(n - 1) {
29         bool updated = false;
30         for (ll u = 0; u <= n; ++u) {
31             for (const auto& [w, v] : adj[u]) {
32                 if (dis[u] < INF and dis[u] + w <
33                     dis[v]) {
34                     dis[v] = dis[u] + w;
35                     updated = true;
36                 }
37             }
38             if (not updated) break;
39         }
40
41         bool hasNegativeCycle = false;
42         for (ll u = 0; not hasNegativeCycle and u
43             <= n; ++u) {
44             for (const auto& [w, v] : adj[u]) {
45                 if (dis[u] < INF and dis[u] + w < dis[
46                     v]) {
47                     hasNegativeCycle = true;
48                     break;
49                 }
50             }
51         }
52         return 0;
53     }

```

6.4 Topo-sort Kahn

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     ll n, m; cin >> n >> m;
6     vector<ll> indeg(n + 1, 0);
7     vector<vector<ll>> adj(n + 1);
8     rep(m) {
9         ll u, v; cin >> u >> v;
10        adj[u].pb(v);
11        indeg[v] += 1;
12    }
13
14    vector<ll> res; res.reserve(n);

```

```

15 queue<ll> q; // you can use any pool data
16             // structure here
17 for (ll u = 1; u <= n; ++u) {
18     if (indeg[u] == 0) {
19         q.push(u);
20         res.pb(u);
21     }
22 }
23 while (not q.empty()) {
24     ll u = q.front(); q.pop();
25     for (ll v : adj[u]) {
26         indeg[v] -= 1;
27         if (indeg[v] == 0) {
28             q.push(v);
29             res.pb(v);
30         }
31     }
32 }
33 // if (res.size() < n) // there exists a
34 // directed cycle
35
36 return 0;

```

6.5 AP

```

1 void dfs(ll u, ll pa = -1) {
2     ll ch = 0; // 紀錄子節點樹
3     low[u] = in[u] = ++t;
4     for (ll v : adj[u]) {
5         if (v == pa) continue;
6         if (in[v] == -1) {
7             ch += 1; // 子節點數加一
8             dfs(v, u);
9             low[u] = min(low[u], low[v]);
10            if (pa != -1 and low[v] >= in[u]
11                ]) "u is AP" // find AP
12        } else if (in[v] < in[u]) {
13            low[u] = min(low[u], in[v]);
14        }
15        if (pa == -1 and ch >= 2) "u is AP" //
16        // root 額外判斷
17    }

```

6.6 Dijkstra

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     const ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep(m) {
10        ll u, v, w; cin >> u >> v >> w;
11        adj[u].pb(pll(w, v));

```

```

12 }
13
14 vector<ll> dis(n + 1, INF);
15 priority_queue<pll, vector<pll>, greater<
16     pll>> pq;
17
18 dis[1] = 0;
19 pq.push(pll(0, 1));
20 while (not pq.empty()) {
21     auto [uw, u] = pq.top(); pq.pop();
22
23     if (uw > dis[u]) continue;
24
25     for (auto [w, v] : adj[u]) {
26         if (dis[u] + w < dis[v]) {
27             dis[v] = dis[u] + w;
28             pq.push(pll(dis[v], v));
29         }
30     }
31 }

```

6.7 Contracting Vertices Based on Specific Conditions

```

1 // define a DSU first
2
3 ll v, e; cin >> v >> e; // the number of
4 // vertices and edges
5
6 vector<pll> edges;
7 rep(e) {
8     ll x, y; cin >> x >> y; // bidirectional
9     // edge (x, y)
10    if (fit_some_condition) join(x, y);
11    else edges.pb(pll(x, y));
12 }
13
14 vector<vector<ll>> adj; // undirected graph
15 for (const pll& edge : edges) {
16     ll x = find_root(edge.ff), y = find_root(
17         edge.ss);
18     adj[x].pb(y);
19     adj[y].pb(x);
20 }

```

6.8 MST Prim

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     constexpr ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep(m) {
10        ll u, v, w; cin >> u >> v >> w;
11        adj[u].pb(pll(w, v));

```

```

12 adj[v].pb(pll(w, u));
13 }
14
15 priority_queue<pll, vector<pll>, greater<
16     pll>> pq;
17 vector<bool> vis(n + 1, false);
18 vector<ll> curw(n + 1, INF);
19
20 pq.push(pll(0, 1));
21 curw[1] = 0;
22
23 ll mstCost = 0, mstEdgesCnt = 0;
24 while (mstEdgesCnt < n - 1) {
25     if (pq.empty()) {
26         // the graph is disconnected (
27         // MST D.N.E.)
28         return 0;
29     }
30
31     auto [uw, u] = pq.top(); pq.pop();
32
33     if (uw > curw[u]) continue;
34
35     vis[u] = true;
36     mstCost += uw;
37     mstEdgesCnt += (u == 1 ? 0 : 1);
38
39     for (const auto& [w, v] : adj[u]) {
40         if (not vis[v] and w < curw[v]) {
41             pq.push(pll(w, v));
42             curw[v] = w;
43         }
44     }
45
46     return 0;
47 }

```

6.9 Dinic

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 constexpr ll INF = 1e18;
6
7 struct Node {
8     ll nd, pvIdx, cap;
9     Node(ll _nd, ll _pvIdx, ll _cap) : nd(
10         _nd), pvIdx(_pvIdx), cap(_cap) {}
11 };
12
13 ll n, m, st, en;
14 vector<ll> level, iter;
15 vector<vector<Node>> adj;
16
17 void bfs() {
18     level.clear(); level.resize(n + 1, -1);
19     level[st] = 0;
20
21     queue<ll> q; q.push(st);
22     while (not q.empty()) {

```

```

22     ll sz = q.size();
23     for (ll _ = 1; _ <= sz; ++_) {
24         ll u = q.front(); q.pop();
25         for (const Node& v : adj[u]) {
26             if (v.cap > 0 and level[v.nd]
27                 == -1) {
28                 level[v.nd] = level[u] +
29                     1;
30                 q.push(v.nd);
31             }
32         }
33     }
34 }
35 ll dfs(ll u, ll flow) {
36     if (u == en) return flow;
37     for (ll& i = iter[u]; i < adj[u].size();
38         ++i) {
39         Node& v = adj[u][i];
40         if (v.cap > 0 and level[v.nd] ==
41             level[u] + 1) {
42             ll fw = dfs(v.nd, min(flow, v.
43                 cap));
44             if (fw > 0) {
45                 v.cap -= fw;
46                 adj[v.nd][v.pvIdx].cap += fw
47                 ;
48                 return fw;
49             }
50         }
51     }
52     return 0; // for compile warning
53 }
54 ll maxFlow() {
55     ll res = 0;
56     while (true) {
57         bfs();
58         if (level[en] == -1) break;
59         ll cur;
60         iter.clear(); iter.resize(n + 1);
61         while ((cur = dfs(st, INF)) > 0) {
62             res += cur;
63         }
64     }
65     return res;
66 }
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70     cin >> n >> m;
71     adj.clear(); adj.resize(n + 1);
72     for (ll i = 1; i <= m; ++i) {
73         ll u, v, cap; cin >> u >> v >> cap;
74         adj[u].pb(Node(v, adj[v].size(), cap));
75         adj[v].pb(Node(u, adj[u].size() - 1,
76             0));
77     }
78     st = 1;
79     en = n;
80     cout << maxFlow() << '\n';

```

6.10 Tarjan 2-SAT

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }
34 }
35 int main() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38     ll x, y; cin >> x >> y;
39     n = 2 * y;
40     inStack.resize(n + 1);
41     in.resize(n + 1, -1);
42     low.resize(n + 1);
43     scc.resize(n + 1);
44     adj.resize(n + 1);
45     for (ll i = 1; i <= x; ++i) {
46         char c1, c2;
47         ll a, b;
48         cin >> c1 >> a >> c2 >> b;
49
50         if (c1 == '+' ) {
51             adj[a + y].pb((c2 == '+' ? b : b
52                 + y));
53         } else {
54             adj[a].pb((c2 == '+' ? b : b + y
55                 ));

```

```

56     }
57
58     if (c2 == '+' ) {
59         adj[b + y].pb((c1 == '+' ? a : a +
60             y));
61     } else {
62         adj[b].pb((c1 == '+' ? a : a + y
63             ));
64     }
65 }
66 for (ll u = 1; u <= n; ++u) {
67     if (in[u] != -1) continue;
68     dfs(u);
69 }
70 bool valid = true;
71 string s = "";
72 for (ll u = 1; u <= y; ++u) {
73     if (scc[u] == scc[u + y]) {
74         valid = false;
75         break;
76     } else if (scc[u] < scc[u + y]) {
77         s += "+ ";
78     } else {
79         s += "- ";
80     }
81 }
82 cout << (valid ? s : "IMPOSSIBLE") << '\n';
83
84 return 0;
85 }

```

6.11 Topo-sort DFS

```

1 ll v;
2 vector<ll> topo;
3 vector<bool> vis;
4 vector<vector<ll>> adj(v + 1);
5 void dfs(ll node) {
6     vis[node] = true;
7     for (ll neighbor : adj[node]) {
8         if (vis[neighbor]) continue;
9         dfs(neighbor);
10    }
11    topo.pb(node);
12 }
13 reverse(topo.begin(), topo.end());

```

6.12 Dinic 無權二分圖最大匹配

```

1 // 輸出無權二分圖最大匹配的邊數量和選到的邊
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 using ll = long long;
6 #define pb push_back
7 constexpr ll INF = 1e18;

```

```

8 struct Node {
9     ll nd, pvIdx, cap;
10     Node(ll _nd, ll _pvIdx, ll _cap) : nd(_nd), pvIdx(_pvIdx), cap(_cap) {}
11 };
12
13 ll n1, n2, n, m, st, en;
14 vector<ll> level, iter;
15 vector<vector<Node>> adj;
16
17 void bfs() {
18     level.clear(); level.resize(n + 1, -1);
19     level[st] = 0;
20
21     queue<ll> q; q.push(st);
22     while (not q.empty()) {
23         ll sz = q.size();
24         for (ll _ = 1; _ <= sz; ++_) {
25             ll u = q.front(); q.pop();
26             for (const Node& v : adj[u]) {
27                 if (v.cap > 0 and level[v.nd]
28                     == -1) {
29                     level[v.nd] = level[u] +
30                         1;
31                     q.push(v.nd);
32                 }
33             }
34         }
35     }
36 }
37 ll dfs(ll u, ll flow) {
38     if (u == en) return flow;
39     for (ll& i = iter[u]; i < adj[u].size();
40         ++i) {
41         Node& v = adj[u][i];
42         if (v.cap > 0 and level[v.nd] ==
43             level[u] + 1) {
44             ll fw = dfs(v.nd, min(flow, v.
45                 cap));
46             if (fw > 0) {
47                 v.cap -= fw;
48                 adj[v.nd][v.pvIdx].cap += fw
49                 ;
50                 return fw;
51             }
52         }
53     }
54     return 0; // for compile warning
55 }
56 ll maxFlow() {
57     ll res = 0;
58     while (true) {
59         bfs();
60         if (level[en] == -1) break;
61         ll cur;
62         iter.clear(); iter.resize(n + 1);
63         while ((cur = dfs(st, INF)) > 0) {
64             res += cur;
65         }
66     }
67     return res;
68 }

```



```

67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     cin >> n1 >> n2 >> m;
72     n = n1 + n2 + 2;
73     st = n1 + n2 + 1;
74     en = n1 + n2 + 2;
75     adj.clear(); adj.resize(n + 1);
76     for (ll i = 1; i <= m; ++i) {
77         ll u, v; cin >> u >> v; v += n1;
78         adj[u].pb(Node(v, adj[v].size(), 1))
79         ;
80         adj[v].pb(Node(u, adj[u].size() - 1, 0));
81     }
82     m += n1 + n2;
83     for (ll i = 1; i <= n1; ++i) {
84         adj[st].pb(Node(i, adj[i].size(), 1)
85         );
86         adj[i].pb(Node(st, adj[st].size() - 1, 0));
87     }
88     for (ll i = n1 + 1; i <= n1 + n2; ++i) {
89         adj[i].pb(Node(en, adj[en].size(), 1));
90         adj[en].pb(Node(i, adj[i].size() - 1, 0));
91     }
92     cout << maxFlow() << '\n';
93     for (ll u = 1; u <= n1; ++u) {
94         for (const Node& v : adj[u]) {
95             if (v.nd != st and v.cap == 0) {
96                 cout << u << ' ' << v.nd -
97                 n1 << '\n';
98             }
99         }
100     }
101     return 0;

```

6.13 Dinic Min Cut

```

1 // 最小割是選一些邊可以阻斷所有水流且這些邊
2 // 的容量和最小
3 // 輸出最小割的容量和選到的邊
4 // (最大流容量和 == 最小割容量和)
5 #include <bits/stdc++.h>
6 using namespace std;
7 using ll = long long;
8 #define pb push_back
9 constexpr ll INF = 1e18;
10
11 struct Node {
12     bool isPseudo;
13     ll nd, pvIdx, cap;
14     Node(ll _nd, ll _pvIdx, ll _cap, bool
15         _isPseudo) : nd(_nd), pvIdx(_pvIdx),
16         cap(_cap), isPseudo(_isPseudo) {}

```

```

15 };
16 ll n, m, st, en;
17 vector<ll> level, iter;
18 vector<vector<Node>> adj;
19
20 void bfs() {
21     level.clear(); level.resize(n + 1, -1);
22     level[st] = 0;
23     queue<ll> q; q.push(st);
24     while (not q.empty()) {
25         ll sz = q.size();
26         for (ll _ = 1; _ <= sz; ++_) {
27             ll u = q.front(); q.pop();
28             for (const Node& v : adj[u]) {
29                 if (v.cap > 0 and level[v.nd]
30                 == -1) {
31                     level[v.nd] = level[u] +
32                     1;
33                     q.push(v.nd);
34                 }
35             }
36         }
37     }
38 }
39
40 ll dfs(ll u, ll flow) {
41     if (u == en) return flow;
42     for (ll& i = iter[u]; i < adj[u].size();
43         ++i) {
44         Node& v = adj[u][i];
45         if (v.cap > 0 and level[v.nd] ==
46             level[u] + 1) {
47             ll fw = dfs(v.nd, min(flow, v.
48             cap));
49             if (fw > 0) {
50                 v.cap -= fw;
51                 adj[v.nd][v.pvIdx].cap += fw;
52                 return fw;
53             }
54         }
55     }
56     return 0; // for compile warning
57 }
58
59 ll maxFlow() {
60     ll res = 0;
61     while (true) {
62         bfs();
63         if (level[en] == -1) break;
64         ll cur;
65         iter.clear(); iter.resize(n + 1);
66         while ((cur = dfs(st, INF)) > 0) {
67             res += cur;
68         }
69     }
70     return res;
71 }
72 vector<bool> vis;
73 void markSetsS(ll u) {
74     vis[u] = true;
75     for (const Node& v : adj[u]) {

```

```

75         if (v.isPseudo) continue;
76         if (not vis[v.nd] and v.cap > 0) {
77             markSetsS(v.nd);
78         }
79     }
80 }
81
82 void outputCut() {
83     for (ll u = 1; u <= n; ++u) {
84         if (not vis[u]) continue; // set T
85         for (const Node& v : adj[u]) {
86             if (v.isPseudo) continue;
87             if (not vis[v.nd]) {
88                 cout << u << ' ' << v.nd <<
89                 '\n';
90             }
91         }
92     }
93 }
94
95 int main() {
96     ios::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99     cin >> n >> m;
100     adj.clear(); adj.resize(n + 1);
101     for (ll i = 1; i <= m; ++i) {
102         ll u, v; cin >> u >> v;
103         adj[u].pb(Node(v, adj[v].size(), 1,
104             false));
105         adj[v].pb(Node(u, adj[u].size() - 1,
106             0, true));
107     }
108     adj[v].pb(Node(u, adj[u].size(), 1,
109         false));
110     adj[u].pb(Node(v, adj[v].size() - 1,
111         0, true));
112 }
113
114 st = 1;
115 en = n;
116 cout << maxFlow() << '\n'; // maxFlow
117 == minCut
118
119 vis.resize(n + 1);
120 markSetsS(st);
121 outputCut();
122
123 return 0;

```

6.14 Floyd Warshall

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     cin >> n >> m;
6     mtx.clear(); mtx.resize(n + 1, vector<ll>(
7     n + 1, INF));
8     for (ll i = 1; i <= n; ++i) mtx[i][i] = 0;
9     rep (m) {
10         ll x, y, w; cin >> x >> y >> w;

```

```

10     mtx[x][y] = min(mtx[x][y], w);
11     mtx[y][x] = min(mtx[y][x], w);
12 } // "min" is used to prevent the
13 // multiple edges
14
15 for (ll k = 1; k <= n; ++k) {
16     for (ll i = 1; i <= n; ++i) {
17         for (ll j = 1; j <= n; ++j) {
18             mtx[i][j] = min(mtx[i][j], mtx[i][k]
19             + mtx[k][j]);
20         }
21     }
22 }

```

6.15 MST Kruskal

```

1 vector<ll> djs, sz;
2 ll find(ll u) {...}
3 void join(ll u, ll v) {...}
4 void init(ll n) {...}
5
6 class Edge{
7 public:
8     ll u, v, w;
9 };
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     // nodes are 1-indexed
16     // edges are 0-indexed
17
18     ll n, m; cin >> n >> m;
19     vector<Edge> edges(m);
20     for (auto& [u, v, w] : edges) {
21         cin >> u >> v >> w;
22     }
23     sort(all(edges), [](const Edge& e1,
24         const Edge e2) -> bool {
25         return e1.w < e2.w;
26     });
27     init(n);
28
29     ll mstCost = 0, mstEdgesCnt = 0;
30     for (const auto& [u, v, w] : edges) {
31         if (find(u) == find(v)) continue;
32         join(u, v);
33         mstCost += w;
34         mstEdgesCnt += 1;
35         if (mstEdgesCnt == n - 1) break;
36     }
37
38     // if (mstEdgesCnt < n - 1) // the graph
39     // is disconnected (MST D.N.E.)
40
41     return 0;

```

6.16 Tarjan 縮點

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx = 0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc, val;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u], low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u], low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }
34 }
35
36 vector<bool> vis2;
37 vector<ll> val2, dp2;
38 vector<vector<ll>> adj2;
39 void dfs2(ll u) {
40     vis2[u] = true;
41     for (ll v : adj2[u]) {
42         if (not vis2[v]) dfs2(v);
43         dp2[u] = max(dp2[u], val2[u] + dp2[v]);
44     }
45 }
46
47 int main() {
48     ios::sync_with_stdio(false);
49     cin.tie(nullptr);
50
51     cin >> n >> m;
52     inStack.resize(n + 1);
53     in.resize(n + 1, -1);
54     low.resize(n + 1);
55     scc.resize(n + 1);
56     val.resize(n + 1);
57     adj.resize(n + 1);
58     for (ll u = 1; u <= n; ++u) cin >> val[u];
59     for (ll i = 1; i <= m; ++i) {
60         ll u, v; cin >> u >> v;
61         adj[u].pb(v);

```

```

62     }
63
64     for (ll u = 1; u <= n; ++u) {
65         if (in[u] != -1) continue;
66         dfs(u);
67     }
68
69     vis2.resize(sccCnt + 1);
70     val2.resize(sccCnt + 1);
71     dp2.resize(sccCnt + 1);
72     adj2.resize(sccCnt + 1);
73     for (ll u = 1; u <= n; ++u) {
74         val2[scc[u]] += val[u];
75         for (ll v : adj[u]) {
76             if (scc[v] == scc[u]) continue;
77             adj2[scc[u]].pb(scc[v]);
78         }
79     }
80     for (ll u = 1; u <= sccCnt; ++u) {
81         dp2[u] = val2[u];
82     }
83
84     for (ll u = 1; u <= sccCnt; ++u) {
85         if (vis2[u]) continue;
86         dfs2(u);
87     }
88
89     cout << *max_element(dp2.begin() + 1,
90                          dp2.end()) << '\n';
91
92     return 0;

```

6.17 Kosaraju

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 #define rep(n) for (ll _ = 1; _ <= n; ++_)
6
7 ll V, E;
8
9 stack<ll> stk;
10 vector<bool> vis;
11 vector<vector<ll>> adj;
12 void dfs1(ll u) {
13     vis[u] = true;
14     for (ll v : adj[u]) {
15         if (vis[v]) continue;
16         dfs1(v);
17     }
18     stk.push(u);
19 }
20
21 ll sccCnt;
22 vector<ll> scc;
23 vector<vector<ll>> radj;
24 void dfs2(ll u, ll sccIdx) {
25     scc[u] = sccIdx;
26     for (ll v : radj[u]) {
27         if (scc[v] != -1) continue;
28         dfs2(v, sccIdx);

```

```

29     }
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     cin >> V >> E;
37     vis.resize(V + 1, false);
38     adj.resize(V + 1);
39     radj.resize(V + 1);
40     rep(E) {
41         ll x, y; cin >> x >> y;
42         adj[x].pb(y);
43         radj[y].pb(x);
44     }
45
46     for (ll u = 1; u <= V; ++u) {
47         if (vis[u]) continue;
48         dfs1(u);
49     }
50
51     sccCnt = 0;
52     scc.resize(V + 1, -1);
53     while (not stk.empty()) {
54         ll u = stk.top(); stk.pop();
55         if (scc[u] != -1) continue;
56         dfs2(u, ++sccCnt);
57     }
58
59     cout << sccCnt << '\n';
60     for (ll i = 1; i <= V; ++i) {
61         cout << scc[i] << ' ';
62     } cout << '\n';
63
64     return 0;
65 }

```

7 Math

7.1 Fast Exponentiation

```

1 ll fastPow(ll b, ll e) {
2     b = b%MOD;
3     ll res = 1;
4     while (e > 0) {
5         if (e%2 == 1) res = (res*b)%MOD;
6         b = (b*b)%MOD;
7         e /= 2;
8     }
9     return res;
10 }

```

7.2 Combinations

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

```

4 const int M = 1e9 + 7;
5 const int N = 2000005;
6 vector<int> fact(N), invfact(N);
7
8 int qpow(int a, int b){
9     int res=1;
10    while(b > 0){
11        if(b & 1) res = res * a % M;
12        a = a * a % M;
13        b >>= 1;
14    }
15    return res;
16 }
17
18 void dofact(){
19     fact[0] = 1;
20     for (int i = 1; i < N; i++) {
21         fact[i] = fact[i - 1] * i % M;
22     }
23
24     invfact[N - 1] = qpow(fact[N - 1], M -
25                          2);
26     for(int i = N - 1; i > 0; i--) invfact[i]
27     - 1] = invfact[i] * i % M;
28 }
29
30 int C(int n,int m){
31     return fact[n] * invfact[m] % M *
32     invfact[n - m] % M;
33 }
34
35 void solve() {
36 }
37
38 signed main() {
39     // FIO;
40     int t_t = 1;
41     // cin >> t_t;
42     while (t_t--){
43         solve();
44     }
45
46     //
47     //
48     // =====
49
50     ll MAXN;
51
52     vector<ll> fact; // factorial
53     vector<ll> invs; // modular inverse
54     void init() {
55         fact.resize(MAXN + 1, 1);
56         invs.resize(MAXN + 1, 1);
57         for (ll i = 2; i <= MAXN; ++i) {
58             fact[i] = (fact[i - 1]*i)%MOD;
59             invs[i] = fastPow(fact[i], MOD - 2);
60         }
61     }
62     inline ll C(ll n, ll k) {

```

```
63 return fact[n]*invs[k]%MOD*invs[n - k]%MOD
64 }
```

7.3 Matrix

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Matrix {
5     int n, m; // n 行, m 列
6     vector<vector<long long>>> a;
7     static const long long MOD = 1e9+7; //
8     // 如果題目需要取模, 可以改這裡
9
10    Matrix(int n, int m, bool ident = false)
11        : n(n), m(m) {
12        a.assign(n, vector<long long>(m, 0))
13        ;
14        if(ident) { // 單位矩陣
15            for(int i=0; i<min(n,m); i++) a[
16                i][i] = 1;
17        }
18    }
19
20    // 輸出矩陣
21    void print() const {
22        for(int i=0; i<n; i++) {
23            for(int j=0; j<m; j++) cout << a
24                [i][j] << " ";
25            cout << "\n";
26        }
27    }
28
29    // 矩陣乘法
30    Matrix operator*(const Matrix& o) const
31    {
32        assert(m == o.n);
33        Matrix res(n, o.m);
34        for(int i=0; i<n; i++) {
35            for(int k=0; k<m; k++) if(a[i][k
36                ]) {
37                for(int j=0; j<o.m; j++) {
38                    res.a[i][j] = (res.a[i][
39                        j] + a[i][k] * o.a[k
40                            ][j]) % MOD;
41                }
42            }
43        }
44        return res;
45    }
46
47    // 矩陣加法
48    Matrix operator+(const Matrix& o) const
49    {
50        assert(n == o.n && m == o.m);
51        Matrix res(n, m);
52        for(int i=0; i<n; i++) {
53            for(int j=0; j<m; j++) {
54                res.a[i][j] = (a[i][j] + o.a
55                    [i][j]) % MOD;
56            }
57        }
58    }
```

```
47 return res;
48 }
49
50 // 矩陣快速幂 (n*n 方陣才能做)
51 Matrix pow(long long exp) const {
52     assert(n == m);
53     Matrix res(n, n, true), base = *this
54     ;
55     while(exp > 0) {
56         if(exp & 1) res = res * base;
57         base = base * base;
58         exp >>= 1;
59     }
60     return res;
61 }
```

7.4 Modular Inverse

```
1 // extend Euclidean
2 pll extgcd(ll a, ll b) {
3     if (b == 0) return pll(1, 0);
4     pll p = extgcd(m, a%b);
5     ll x = p.ff, y = p.ss;
6     return pll(y, x - y*(a/b));
7 }
8 extgcd(a, MOD).ff // the modular inverse
9 (extgcd(a, MOD).ff%MOD + MOD)%MOD // if you
10 want to ensure that it is non-negative
11 // fast exponentiation (make sure that MOD
12 is a prime number)
13 fastPow(a, MOD - 2) // the modular inverse
```

7.5 Big Integer Addition and Multi- plication

```
1 vector<int> strToVec(string str, int sz) {
2     vector<int> r(sz, 0);
3     int strLength = str.length();
4     for (int i = strLength - 1, idx = 0; i
5         >= 0; --i, ++idx) {
6         r[idx] = str[i] - '0';
7     }
8     return r;
9 }
10 // for example:
11 // strToVec("677", 4) -> 7 7 6 0
12 // strToVec("8829", 4) -> 9 2 8 8
13
14 // addition
15 string add(string x, string y) {
16     ll n = max(x.length(), y.length());
17     vector<ll> xdigit = strToVec(x, n + 1);
18     vector<ll> ydigit = strToVec(y, n + 1);
19     vector<ll> result(n + 1, 0);
20     ll carry = 0;
21     for (ll i = 0; i < n + 1; ++i) {
```

```
21 result[i] = xdigit[i] + ydigit[i] +
22     carry;
23 if (result[i] >= 10) {
24     result[i] %= 10;
25     carry = 1;
26 } else carry = 0;
27 }
28 ll start;
29 for (ll i = n; i >= 0; --i) {
30     if (result[i] != 0) {
31         start = i;
32         break;
33     }
34 }
35 string r = "";
36 for (ll i = start; i >= 0; --i) {
37     r += result[i] + '0';
38 }
39 return r;
40 }
41
42 // multiplication
43 string product(string x, string y) {
44     ll xlength = x.length();
45     ll ylength = y.length();
46     ll n = max(xlength, ylength);
47     vector<ll> xdigit = strToVec(x, xlength)
48     ;
49     vector<ll> ydigit = strToVec(y, ylength)
50     ;
51     vector<ll> result(2*n, 0);
52     for (ll i = 0; i < xlength; ++i) {
53         for (ll j = 0; j < ylength; ++j) {
54             result[i + j] += xdigit[i]*
55                 ydigit[j];
56             if (result[i + j] >= 10) {
57                 result[i + j + 1] += result[
58                     i + j]/10;
59                 result[i + j] %= 10;
60             }
61         }
62     }
63     ll start;
64     for (ll i = 2*n - 1; i >= 0; --i) {
65         if (result[i] != 0) {
66             start = i;
67             break;
68         }
69     }
70     string r = "";
71     for (ll i = start; i >= 0; --i) {
72         r += result[i] + '0';
73     }
74     return r;
75 }
```

7.6 Prime Sieve

```
1 // Sieve of Eratosthenes
2 ll MAXN;
3 vector<ll> spf(MAXN + 1, -1);
4 spf[0] = -2; spf[1] = -2;
```

```
5 for (ll i = 2; i*i <= MAXN; ++i) {
6     if (spf[i] == -1) {
7         for (ll j = i*i; j <= MAXN; j += i)
8             if (spf[j] == -1) {
9                 spf[j] = i;
10            }
11        }
12    }
13 }
```

8 Others

8.1 mt19937 Usage

```
1 #include <chrono>
2 #include <random>
3 using namespace std;
4 using ll = long long;
5
6 // mt19937 is a random number generator
7 // chrono::steady_clock::now().
8 // time_since_epoch().count() is used as a
9 // seed
10 mt19937 rng(chrono::steady_clock::now().
11     time_since_epoch().count());
12
13 // dist describes a range of random numbers
14 uniform_int_distribution<ll> dist(lb, ub);
15
16 dist(rng) // use rng to generate a random
17 integer in [lb, ub]
```

8.2 GCC Builtin Functions

```
1 // count the number of 1 bit in x
2 __builtin_popcount(unsigned int x)
3 __builtin_popcountll(unsigned long long x)
4
5 // count leading zero of x
6 __builtin_clz(unsigned int x)
7 __builtin_clzll(unsigned long long x)
8
9 // count trailing zero of x
10 __builtin_ctz(unsigned int x)
11 __builtin_ctzll(unsigned long long x)
```

8.3 distinct k number

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define all(c) (c).begin(), (c).end()
5
6 void solve() {
```

```

7  int n, k, L, R;
8  cin >> n >> k >> L >> R;
9  vector<int> uni(n);
10 vector<int> vec(n);
11 for (int i = 0; i < n; i++) {
12     cin >> uni[i];
13     vec[i] = uni[i];
14 }
15 sort(all(uni));
16 uni.resize(unique(all(uni)) - uni.begin
17 ());
18 vector<int> f_low(uni.size(), 0);
19 vector<int> f_up(uni.size(), 0);
20 for (int i = 0; i < n; i++) {
21     vec[i] = lower_bound(all(uni), vec[i]
22 ) - uni.begin();
23 }
24 int l = -1, r = 0;
25 int cl = 0, cr = 0;
26 int res = 0;
27 for (int i = 0; i < n; i++) {
28     while (r < n && cr <= k) {
29         if (f_up[vec[r]] == 0) {
30             if (cr == k) break;
31             cr++;
32         }
33         f_up[vec[r++]]++;
34     }
35     while (l + 1 < n && cl < k) {
36         if (f_low[vec[l + 1]] == 0) {
37             cl++;
38         }
39         f_low[vec[l + 1]]++;
40         l++;
41     }
42     if (cl == k) {
43         res += max(min(r, i + R) - max(l
44 , i + L - 1), 0LL);
45     }
46     f_low[vec[i]]--;
47     f_up[vec[i]]--;
48     if (f_low[vec[i]] == 0) cl--;
49     if (f_up[vec[i]] == 0) cr--;
50 }
51 cout << res << '\n';

```

```

12     return x ^ (x >> 31);
13 }
14 public:
15     size_t operator()(uint64_t x) const {
16         static const uint64_t FIXED_RANDOM =
17             std::chrono::steady_clock::now
18             ().time_since_epoch().count();
19         return splitmix64(x + FIXED_RANDOM);
20     }
21 };
22 // for example:
23 // unordered_map<LL, LL, custom_hash> mp;
24 // =====
25 // =====
26 class custom_hash {
27 private:
28     static const uint64_t FIXED_RANDOM;
29     static uint64_t splitmix64(uint64_t x) {
30         x += 0x9e3779b97f4a7c15;
31         x = (x ^ (x >> 30)) * 0
32             xbf58476d1ce4e5b9;
33         x = (x ^ (x >> 27)) * 0
34             x94d049bb133111eb;
35         return x ^ (x >> 31);
36     }
37 public:
38     size_t operator()(uint64_t x) const {
39         return splitmix64(x + FIXED_RANDOM);
40     }
41     size_t operator()(pair<uint64_t,
42         uint64_t> p) const {
43         uint64_t h1 = splitmix64(p.first +
44             FIXED_RANDOM);
45         uint64_t h2 = splitmix64(p.second +
46             FIXED_RANDOM + 1);
47         return h1 ^ (h2 * 47);
48     }
49 };
50 const uint64_t custom_hash::FIXED_RANDOM =
51     std::chrono::steady_clock::now().
52     time_since_epoch().count();
53 // for example:
54 // unordered_map<pLL, LL, custom_hash> mp;

```

9 String

9.1 Z-value

```

1 #include <chrono>
2
3 // =====
4 // =====
5
6 class custom_hash {
7 private:
8     static uint64_t splitmix64(uint64_t x) {
9         x += 0x9e3779b97f4a7c15;
10        x = (x ^ (x >> 30)) * 0
11            xbf58476d1ce4e5b9;
12        x = (x ^ (x >> 27)) * 0
13            x94d049bb133111eb;

```

```

1 // CSES: String Matching
2 // Given a string and a pattern, your task
3 // is to count
4 // the number of positions where the pattern
5 // occurs in the string.
6
7 #include <bits/stdc++.h>
8 using namespace std;
9 using ll = long long;
10
11 vector<ll> z_value(const string& s) {

```

```

10 ll n = s.size();
11 vector<ll> z(n);
12 ll l = 0, r = 0;
13 for (ll i = 1; i < n; ++i) {
14     if (i <= r) z[i] = min(z[i - l], r -
15         i + 1);
16     while (i + z[i] < n && s[z[i]] == s[
17         i + z[i]]) z[i] += 1;
18     if (i + z[i] - 1 > r) {
19         l = i;
20         r = i + z[i] - 1;
21     }
22 }
23 return z;
24 }
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     string s1, s2; cin >> s1 >> s2;
30     ll n = s1.size(), m = s2.size();
31
32     ll cnt = 0;
33     string s = s2 + "$" + s1;
34     vector<ll> z = z_value(s);
35     for (ll i = m; i < s.size(); ++i) {
36         if (z[i] == m) cnt += 1;
37     }
38     cout << cnt << '\n';
39
40     return 0;
41 }
42 // =====
43 // =====
44 // CSES: Finding Borders
45 // A border of a string is a prefix that is
46 // also a suffix of
47 // the string but not the whole string. For
48 // example,
49 // the borders of abcababab are ab and
50 // abcab.
51 // Your task is to find all border lengths
52 // of a given string.

```

```

53 #include <bits/stdc++.h>
54 using namespace std;
55 using ll = long long;
56 #define pb push_back
57
58 vector<ll> z_value(const string& s) {
59     ll n = s.size();
60     vector<ll> z(n);
61     ll l = 0, r = 0;
62     for (ll i = 1; i < n; ++i) {
63         if (i <= r) z[i] = min(z[i - l], r -
64             i + 1);
65         while (i + z[i] < n && s[z[i]] == s[
66             i + z[i]]) z[i] += 1;
67         if (i + z[i] - 1 > r) {
68             l = i;
69             r = i + z[i] - 1;
70         }
71     }
72 }

```

```

68     return z;
69 }
70
71 int main() {
72     ios::sync_with_stdio(false);
73     cin.tie(nullptr);
74
75     string s; cin >> s;
76     ll n = s.size();
77
78     vector<ll> z = z_value(s), res;
79     for (ll i = 1; i < n; ++i) {
80         if (i + z[i] == n) res.pb(z[i]);
81     }
82     sort(res.begin(), res.end());
83     for (ll x : res) {
84         cout << x << ' ';
85     } cout << '\n';
86
87     return 0;
88 }
89 // =====
90 // =====
91 // CSES: Finding Periods
92 // A period of a string is a prefix that can
93 // be used to generate
94 // the whole string by repeating the prefix.
95 // The last repetition
96 // may be partial. For example, the periods
97 // of abcbabca are abc, abcbab and abcbabca.
98 // Your task is to find all period lengths
99 // of a string.
100
101 #include <bits/stdc++.h>
102 using namespace std;
103 using ll = long long;
104 #define pb push_back
105
106 vector<ll> z_value(const string& s) {
107     ll n = s.size();
108     vector<ll> z(n);
109     ll l = 0, r = 0;
110     for (ll i = 1; i < n; ++i) {
111         if (i <= r) z[i] = min(z[i - l], r -
112             i + 1);
113         while (i + z[i] < n && s[z[i]] == s[
114             i + z[i]]) z[i] += 1;
115         if (i + z[i] - 1 > r) {
116             l = i;
117             r = i + z[i] - 1;
118         }
119     }
120     return z;
121 }
122
123 int main() {
124     ios::sync_with_stdio(false);
125     cin.tie(nullptr);
126
127     string s; cin >> s;
128     ll n = s.size();
129
130     vector<ll> z = z_value(s), res;
131     for (ll i = 1; i < n; ++i) {

```

8.4 Custom Hash

```

128     if (i + z[i] == n) res.pb(i);
129 }
130 sort(res.begin(), res.end());
131 for (ll x : res) {
132     cout << x << ' ';
133 } cout << n << '\n';
134
135 return 0;
136 }

```

9.2 KMP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 template<class T> using vec = vector<T>;
6 #define endl '\n'
7 #define ff first
8 #define ss second
9 #define pb push_back
10 #define rep(n) for (ll _ = 1; _ <= n; ++_)
11
12 // -----
13
14 vector<ll> LPS(const string& s) { // 0-
15     indexed
16     ll n = s.size();
17     if (n == 0) return {};
18     vector<ll> lps(n); lps[0] = 0;
19     for (ll i = 1, j = 0; i < n; ++i) {
20         while (j > 0 && s[i] != s[j]) j =
21             lps[j - 1];
22         if (s[i] == s[j]) j += 1;
23         lps[i] = j;
24     }
25     return lps;
26 }
27
28 ll KMP(const string& s, const string& t) {
29     // 0-indexed
30     vector<ll> lps = LPS(t);
31     ll n = s.size(), m = t.size();
32     if (n < m) return 0;
33     ll i = 0, j = 0, cnt = 0;
34     while (i < n) {
35         if (s[i] == t[j]) {
36             i += 1;
37             j += 1;
38         } else if (j > 0) {
39             j = lps[j - 1];
40         } else if (j == 0) {
41             i += 1;
42         }
43     }
44     if (j == m) {
45         // s[i - m to i - 1] == t
46         cnt += 1;
47         j = lps[j - 1];
48     }
49 }

```

```

46     return cnt; // the number of matching
47     substrings
48 }
49
50 int main() {
51     ios::sync_with_stdio(false);
52     cin.tie(nullptr);
53
54     string s, t;
55     cin >> s >> t;
56
57     cout << KMP(s, t) << endl;
58
59     return 0;
60 }

```

9.3 Trie

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Trie {
5     struct Node {
6         bool endofWord;
7         vector<int> children;
8         Node() : endofWord(false), children
9             (26, -1) {}
10    };
11
12    vector<Node> trie;
13
14    public:
15    Trie() {
16        trie.emplace_back();
17    }
18
19    void insert(const string& word) {
20        int cur = 0;
21        for (char c : word) {
22            int idx = c - 'a';
23            if (trie[cur].children[idx] ==
24                -1) {
25                trie[cur].children[idx] =
26                    trie.size();
27                trie.emplace_back();
28            }
29            cur = trie[cur].children[idx];
30        }
31        trie[cur].endofWord = true;
32    }
33
34    bool search(const string& word) {
35        int cur = 0;
36        for (char c : word) {
37            int idx = c - 'a';
38            if (trie[cur].children[idx] ==
39                -1) return false;
40            cur = trie[cur].children[idx];
41        }
42        return trie[cur].endofWord;
43    }
44
45    bool startsWith(const string& prefix) {

```

```

46        int cur = 0;
47        for (char c : prefix) {
48            int idx = c - 'a';
49            if (trie[cur].children[idx] ==
50                -1) return false;
51            cur = trie[cur].children[idx];
52        }
53        return true;
54    }
55
56    void deleteWord(const string& word) {
57        int cur = 0;
58        for (char c : word) {
59            int idx = c - 'a';
60            if (trie[cur].children[idx] ==
61                -1) return;
62            cur = trie[cur].children[idx];
63        }
64        trie[cur].endofWord = false;
65    }
66
67    void print(int node, string prefix)
68    const {
69        if (trie[node].endofWord) {
70            cout << prefix << "\n";
71        }
72        for (int i = 0; i < 26; i++) {
73            int nxt = trie[node].children[i];
74            if (nxt != -1) {
75                print(nxt, prefix + char('a'
76                    + i));
77            }
78        }
79    }
80
81    void print() const { print(0, ""); }
82
83    };
84
85    int main() {
86        Trie trie;
87
88        trie.insert("geek");
89        trie.insert("geeks");
90        trie.insert("code");
91        trie.insert("coder");
92        trie.insert("coding");
93
94        cout << "Trie contents:\n";
95        trie.print();
96
97        cout << "\nSearch results:\n";
98        cout << "geek: " << trie.search("geek")
99        << "\n";
100        cout << "geeks: " << trie.search("geeks")
101        << "\n";
102        cout << "code: " << trie.search("code")
103        << "\n";
104        cout << "coder: " << trie.search("coder")
105        << "\n";
106        cout << "coding: " << trie.search("coding")
107        << "\n";
108        cout << "codex: " << trie.search("codex")
109        << "\n";
110
111        cout << "\nPrefix results:\n";

```

```

97     cout << "ge: " << trie.startsWith("ge")
98     << "\n";
99     cout << "cod: " << trie.startsWith("cod")
100     << "\n";
101     cout << "coz: " << trie.startsWith("coz")
102     << "\n";
103
104     trie.deleteWord("coding");
105     trie.deleteWord("geek");
106
107     cout << "\nTrie contents after deletions
108     :\n";
109     trie.print();
110
111     cout << "\nSearch results after
112     deletions:\n";
113     cout << "coding: " << trie.search("coding")
114     << "\n";
115     cout << "geek: " << trie.search("geek")
116     << "\n";
117
118     return 0;
119 }

```

9.4 Manacher

```

1 // CSES: Longest Palindrome
2 // Given a string, your task is to determine
3 // the longest
4 // palindromic substring of the string. For
5 // example,
6 // the longest palindrome in aybabtu is bab.
7
8 #include <bits/stdc++.h>
9 using namespace std;
10 using ll = long long;
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     string t, s = "^#"; cin >> t;
17     ll n = t.size(), m = 2*n + 3;
18     for (ll i = 0; i < n; ++i) {
19         s += t[i];
20         s += (i == n - 1 ? "$" : "#");
21     }
22
23     ll c = 1;
24     vector<ll> p(m); p[1] = 0;
25     for (ll i = 2; i <= m - 3; ++i) {
26         if (i < c + p[c]) p[i] = min(p[c - (i - c)], (c + p[c]) - i);
27         while (i + p[i] + 1 < m && s[i - p[i] - 1] == s[i + p[i] + 1]) p[i] += 1;
28         if (i + p[i] > c + p[c]) c = i;
29     }
30
31     ll j = 2;
32     for (ll i = 3; i <= m - 3; ++i) {
33         if (p[i] > p[j]) j = i;

```

```

34 }
35
36 for (ll i = j - p[j] + 1; i <= j + p[j]
37       - 1; i += 2) {
38     cout << s[i];
39 } cout << '\n';
40
41 return 0;
42 }
43 // =====
44 // =====
45
46 // CSES: All Palindromes
47 // Given a string, calculate for each
48 // position the length
49 // of the longest palindrome that ends at
50 // that position.
51
52 #include <bits/stdc++.h>
53 using namespace std;
54 using ll = long long;
55
56 ll n, m;
57 string ori_s, s;
58 vector<ll> p, rt, dp;
59
60 int main() {
61     ios::sync_with_stdio(false);
62     cin.tie(nullptr);
63
64     cin >> ori_s;
65     n = ori_s.size();
66     m = 2*n + 3;
67     s = "^#";
68     for (ll i = 0; i < n; ++i) {
69         s += ori_s[i];
70         s += (i == n - 1 ? "$" : "#");
71     }
72
73     ll c = 0;
74     p.resize(m);
75     for (ll i = 2; i <= m - 3; ++i) {
76         if (i < c + p[c]) p[i] = min(p[c] - (i - c), (c + p[c]) - i);
77         while (i - p[i] - 1 >= 0 && i + p[i] + 1 < m && s[i - p[i] - 1] == s[i + p[i] + 1]) p[i] += 1;
78         if (i + p[i] > c + p[c]) c = i;
79     }
80
81     rt.resize(n, 1);
82     for (ll i = 2; i <= m - 3; ++i) {
83         ll y = ((i + p[i] - 1) - 2)/2;
84         if (s[i] == '#') {
85             ll x = ((i + 1) - 2)/2;
86             rt[y] = max(rt[y], (y - x + 1) * 2);
87         } else {
88             ll x = (i - 2)/2;
89             rt[y] = max(rt[y], (y - x) * 2 + 1);
90         }
91     }
92 }

```

```

93 dp.resize(n); dp[n - 1] = rt[n - 1];
94 for (ll i = n - 2; i >= 0; --i) {
95     dp[i] = max(rt[i], dp[i + 1] - 2);
96 }
97
98 for (ll x : dp) {
99     cout << x << ' ';
100 } cout << '\n';
101
102 return 0;
103 }

```

9.5 String Hashing

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 class strHash {
8 private:
9     const ll m1 = 1e9 + 7, m2 = 1e9 + 9, p = 239017;
10
11     ll n;
12     string s;
13     vector<ll> h1, h2, p1, p2;
14 public:
15     strHash(const string& _s) {
16         n = _s.size();
17         s = _s;
18         h1.resize(n); h1[0] = s[0];
19         h2.resize(n); h2[0] = s[0];
20         for (ll i = 1; i < n; ++i) {
21             h1[i] = (h1[i - 1]*p%m1 + s[i])%m1;
22             h2[i] = (h2[i - 1]*p%m2 + s[i])%m2;
23         }
24
25         p1.resize(n); p1[0] = 1;
26         p2.resize(n); p2[0] = 1;
27         for (ll i = 1; i < n; ++i) {
28             p1[i] = p1[i - 1]*p%m1;
29             p2[i] = p2[i - 1]*p%m2;
30         }
31     }
32
33     pll hash(ll l, ll r) const { // [l, r]
34         if (l == 0) return pll(h1[r], h2[r]);
35         return pll(
36             ((h1[r] - h1[l - 1]*p1[r - l + 1]%m1 + m1)%m1,
37             ((h2[r] - h2[l - 1]*p2[r - l + 1]%m2 + m2)%m2)
38         );
39     }
40
41 int main() {
42     ios::sync_with_stdio(false);
43     cin.tie(nullptr);

```

```

44
45     string s1, s2; cin >> s1 >> s2;
46     ll n = s1.size(), m = s2.size();
47
48     if (n < m) {
49         cout << 0 << '\n';
50         return 0;
51     }
52
53     ll cnt = 0;
54     strHash sh1(s1), sh2(s2);
55     pll tarHash = sh2.hash(0, m - 1);
56     for (ll i = 0; i < n - m + 1; ++i) {
57         if (sh1.hash(i, i + m - 1) == tarHash) cnt += 1;
58     }
59     cout << cnt << '\n';
60
61     return 0;
62 }

```

9.6 Suffix Array n log squared n version

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 vector<ll> suffix_array(const string& s) {
6     ll n = s.size();
7     vector<ll> sa(n), rk(n), tmp(n);
8
9     for (ll i = 0; i < n; ++i) {
10         sa[i] = i;
11         rk[i] = s[i] - 'a';
12     }
13
14     for (ll k = 1; k < n; k *= 2) {
15         auto cmp = [&](ll ri, ll li) {
16             if (rk[ri] != rk[li]) return rk[ri] < rk[li];
17             ll x = (ri + k < n) ? rk[ri + k] : -1;
18             ll y = (li + k < n) ? rk[li + k] : -1;
19             return x < y;
20         };
21
22         sort(sa.begin(), sa.end(), cmp);
23
24         tmp[sa[0]] = 0;
25         for (ll i = 1; i < n; ++i) {
26             tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i]));
27         }
28         rk = tmp;
29     }
30
31     return sa;
32 }
33
34 int main() {

```

```

35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     string s; cin >> s;
39     vector<ll> sa = suffix_array(s);
40     ll n = s.size();
41
42     ll k; cin >> k;
43     for (ll i = 0; i < k; ++i) {
44         string tar; cin >> tar;
45         ll m = tar.size();
46
47         ll l = 0, r = n - 1;
48         while (l <= r) {
49             ll mid = l + (r - l)/2;
50             if (s.substr(sa[mid], m) < tar) {
51                 l = mid + 1;
52             } else {
53                 r = mid - 1;
54             }
55         }
56
57         cout << (s.substr(sa[l], m) == tar ? "YES" : "NO") << '\n';
58     }
59
60     return 0;
61 }

```

10 Tree

10.1 Binary Lifting

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 inline ll flg(ll x) {
6     return 63 - __builtin_clzll(x);
7 }
8
9 inline bool isOnBit(ll x, ll i) {
10     return ((1LL << i) & x) > 0;
11 }
12
13 ll n, q, lgn;
14 vector<vector<ll>> blf;
15
16 void init() {
17     blf[0][1] = -1;
18     for (ll u = 2; u <= n; ++u) cin >> blf[0][u];
19
20     for (ll h = 1; h <= lgn; ++h) {
21         for (ll u = 1; u <= n; ++u) {
22             ll nt = blf[h - 1][u];
23             blf[h][u] = nt == -1 ? -1 : blf[h - 1][nt];
24         }
25     }

```



```

26 }
27 ll query(ll u, ll step) {
28     ll cur = u;
29     for (ll i = 30; i >= 0; --i) {
30         if (isOnBit(step, i)) {
31             cur = blf[i][cur];
32             if (cur == -1) return -1;
33         }
34     }
35     return cur;
36 }
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     cin >> n >> q;
43     lgn = flg(n);
44     blf.resize(lgn + 1, vector<ll>(n + 1));
45     init();
46
47     while (q--) {
48         ll u, step; cin >> u >> step;
49         cout << query(u, step) << '\n';
50     }
51
52     return 0;
53 }

```

10.2 LCA

```

1 // Use binary Lifting
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 using ll = long long;
6 #define pb push_back
7
8 inline ll flg(ll x) {
9     return 63 - __builtin_clzll(x);
10 }
11
12 inline bool isOnBit(ll x, ll i) {
13     return ((1LL << i) & x) > 0;
14 }
15
16 ll n, q, lgn;
17 vector<ll> d;
18 vector<vector<ll>> blf, adj;
19
20 void init() {
21     blf[0][1] = -1;
22     for (ll u = 2; u <= n; ++u) {
23         ll v; cin >> v;
24         blf[0][u] = v;
25         adj[v].pb(u);
26     }
27
28     for (ll h = 1; h <= lgn; ++h) {
29         for (ll u = 1; u <= n; ++u) {
30             ll nt = blf[h - 1][u];
31             blf[h][u] = nt == -1 ? -1 : blf[

```

```

32         ]
33     }
34 }
35 ll query(ll u, ll step) {
36     ll cur = u;
37     for (ll i = 30; i >= 0; --i) {
38         if (isOnBit(step, i)) {
39             cur = blf[i][cur];
40             if (cur == -1) return -1;
41         }
42     }
43     return cur;
44 }
45
46 void dfs(ll u, ll dn) {
47     d[u] = dn;
48     for (ll v : adj[u]) {
49         dfs(v, dn + 1);
50     }
51 }
52
53 ll lca(ll u, ll v) {
54     if (d[u] > d[v]) swap(u, v);
55     if (d[u] < d[v]) v = query(v, d[v] - d[u]
56         );
57     if (u == v) return u;
58     for (ll h = lgn; h >= 0; --h) {
59         ll ntu = blf[h][u];
60         ll ntv = blf[h][v];
61         if (ntu == -1 or ntv == -1 or ntu ==
62             ntv) continue;
63         u = ntu;
64         v = ntv;
65     }
66     return blf[0][u];
67 }
68
69 int main() {
70     ios::sync_with_stdio(false);
71     cin.tie(nullptr);
72
73     cin >> n >> q;
74     lgn = flg(n);
75     d.resize(n + 1);
76     blf.resize(lgn + 1, vector<ll>(n + 1));
77     adj.resize(n + 1);
78     init();
79     dfs(1, 0);
80
81     while (q--) {
82         ll u, v; cin >> u >> v;
83         cout << lca(u, v) << '\n';
84     }
85
86     return 0;
87 }

```

ACM ICPC Team Reference - c0mpile_error

Contents

1 Bitwise Trick	1	3 D&C	2	6.4 Topo-sort Kahn	7	7.5 Big Integer Addition and Multiplication	11
1.1 tricks	1	3.1 MergeSort Finds the Number of Inversions	2	6.5 AP	7	7.6 Prime Sieve	11
2 DP	1	4 Data Structure	2	6.6 Dijkstra	7	8 Others	11
2.1 LIS	1	4.1 Segment Tree	2	6.7 Contracting Vertices Based on Specific Conditions	7	8.1 mt19937 Usage	11
2.2 Knapsack	1	4.2 BIT	3	6.8 MST Prim	7	8.2 GCC Builtin Functions	11
2.3 Subset DP	1	4.3 Treap	3	6.9 Dinic	7	8.3 distinct k number	11
2.4 Interval DP	1	4.4 Sparse Table	5	6.10 Tarjan 2-SAT	8	8.4 Custom Hash	12
2.5 LCS	1	4.5 DSU	5	6.11 Topo-sort DFS	8	9 String	12
2.6 Digit DP	2	5 Geometry	6	6.12 Dinic 無權二分圖最大匹配	8	9.1 Z-value	12
		5.1 Line Segment Intersection Test	6	6.13 Dinic Min Cut	9	9.2 KMP	13
		5.2 Simple Polygon Area	6	6.14 Floyd Warshall	9	9.3 Trie	13
		5.3 Vector	6	6.15 MST Kruskal	9	9.4 Manacher	13
		5.4 Convex Hull	6	6.16 Tarjan 縮點	10	9.5 String Hashing	14
		6 Graph	6	6.17 Kosaraju	10	9.6 Suffix Array n log squared n version	14
		6.1 Tarjan	6	7 Math	10	10 Tree	14
		6.2 Bridge	6	7.1 Fast Exponentiation	10	10.1 Binary Lifting	14
		6.3 Bellman Ford Detects Nega- tive Cycle	6	7.2 Combinations	10	10.2 LCA	15
				7.3 Matrix	11		
				7.4 Modular Inverse	11		

ACM ICPC Judge Test - c0mpile_error

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```